



WHEATON
COLLEGE

For Christ & His Kingdom

AI for Theological Inquiry

Packaging — from a command line to a chatbot
others can use

Huihuo Zheng
Wheaton College
Fall 2026

huihuo.zheng@wheaton.edu

You built a real RAG system. Nobody else can use it.

>_ What you have

- A retriever over **31,100 verses**
- A grounded prompt that **quotes and cites**
- A model that **refuses** when the text is silent

Genuinely good work — behind a python command.

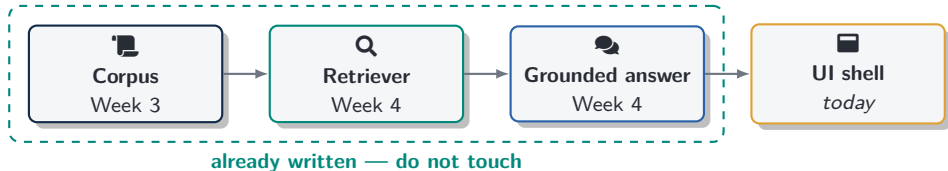
👤 Who your work is for

- A professor reading your project
- A classmate testing your claims
- A pastor or student in your tradition

None of them will clone a repo and read your argv parsing.

Research that cannot be *handled* by another person is hard to examine — and easy to over-trust.

Packaging is thin — that is the whole point



The rule: import it

`chat_bible.py` **imports** `build_retriever / ask_rag / ask_plain` from your own `ask_bible_local.py`. **Zero** lines of retrieval logic are rewritten.

The anti-pattern: copy it

Paste the retrieval code into the UI file and you now have **two** systems that will **disagree** — and the one you evaluated is not the one you demo.

The design rule for *this* course: a UI must show its work

A chatbot hides things

A polished chat bubble looks **equally confident** whether the answer came from the text or from the model's memory.

Packaging can make a system *less* honest than the CLI it replaced.

So ours shows the grounding

Under *every* answer: the **exact verses retrieved**, with scores — plus an optional **ungrounded** answer side by side.

The interface enforces *don't trust, verify*.



This is not decoration. The sources panel *is* your **disclosure appendix**, generated live: what was retrieved, what was answered, and what the model would have said without the text in front of it.

The entire packaging layer, honestly

```
# 1. IMPORT your own RAG system -- no reimplementation
from ask_bible_local import build_retriever, ask_rag, ask_plain

# 2. CACHE the retriever (Streamlit reruns the script on every keystroke)
@st.cache_resource
def load_retriever(use_embeddings):
    return build_retriever(force_embed=use_embeddings)

# 3. RENDER: the answer, then the grounding
answer, hits = ask_rag(question, retriever, k=k)
st.markdown(answer)
with st.expander(f"Sources retrieved ({len(hits)} verses)"):
    for v, score in hits:
        st.markdown(f"**[{v['ref']}]** *(score {score:.3f})* \n{v['text']}")
```

Import · cache · render. If you can read your RAG script, you can read this one.

Hands-on: launch it (~20 min)

```
pip install -r requirements-chatbot.txt # one new dependency: streamlit
python inference_auth_token.py authenticate # if your token expired
streamlit run chat_bible.py # opens http://localhost:8501
```

🔍 Ask these three

1. *Quote John 3:16 exactly.* — grounded quotes; ungrounded paraphrases
2. *What does Micah 6:8 require?*
3. *What does Hezekiah 3:16 say?* — **no such book**: grounded must refuse

⚙️ Then turn the knobs

k from 2 to 10 — watch answers get vaguer as grounding gets diluted.

Flip **BM25** ↔ **embeddings** on a *paraphrase* question (“still small voice”) and on a *citation* question.

Two doors into the same room: Streamlit or Gradio

☰ Streamlit — chat_bible.py

Script reruns top-to-bottom on every interaction. Sidebar + expanders make the **grounding panel** free.

Best when you want controls and a dashboard feel.

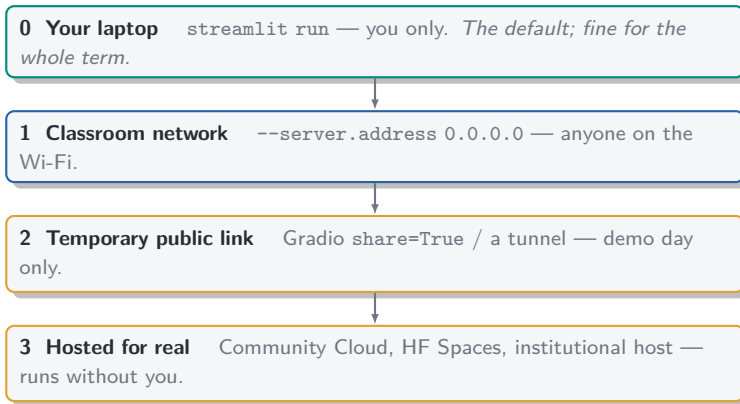
🤖 Gradio — chat_bible_gradio.py

`gr.ChatInterface` gives a chat UI in a few lines; sources ride along in a collapsible block under each answer.

Best when you want the shortest path — and `share=True`.

🔧 Both files **import the identical three functions**. The framework is interchangeable; **your RAG system is the asset**. That is how you know the packaging was thin enough.

Shipping it to other people: a four-rung ladder



Each rung up adds one reachable stranger — and one problem you now own.

The wall you will hit: whose key is answering?

Your ALCF token is *personal*

Browser login · expires in ~48 h · billed to **you**.

On rungs 1–3, every visitor's question is spent **from your account**, under **your name** — and the app dies when the token does.

The honest answer

Rung 3 needs a **service credential**, not your login — which the ALCF student endpoint does not give you.

Stay on rung 0–1; use rung 2 for the demo. A permanent public bot needs a different backend and a real conversation.

 Never paste a token into your code or commit it. `share=True` publishes to **the open internet with no password**. See `DEPLOY.md` for the checklist.

The turn: who is answerable for what it says?



Deploying is a pastoral act

A CLI answers **you**, who know its limits. A public chatbot answers a **stranger in distress at 2 a.m.** — who may read it as counsel.

The same code; a different moral situation.



Minimum decencies

Say **what corpus** it reads and **whose canon** that is. Say it is a **student project**, not a pastor. **Show the sources.** Say what it **will not** do.



Polish is a **truth claim** users can read. The more finished your interface looks, the more your disclosure has to work — accountability should rise **with** usability, not fall behind it.

What to take away & what to do

✓ Three things

1. **Packaging is thin** — import your system, never re-implement it
2. **A good UI shows its work** — sources under every answer
3. **Every rung of deployment adds a person you are answerable to**

To do

- Get the chatbot running on **your own corpus** — retitle it honestly
- Add a **one-line disclosure** in the sidebar: corpus, canon, “student project”
- Bring it to the **build sprint**; we demo on rung 1 or 2

Next: evaluation — now that it looks convincing, how do you prove it is right?